SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI. problems efficiently. (BL3)

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _GURURAMCHARAN P (1925260193_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOL 1

1. AI-Powered Drone Delivery in a Flood-Affected Region

i. Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search chooses the next location based on the heuristic value, which in this case is the straight-line distance to the destination. The drone will usually select the path that appears to bring it closest to the emergency medicine delivery point. Its main advantage is speed because it can focus on promising routes and may find a solution quickly. However, it does not consider the full cost already spent on the route. A short-looking path may contain deep water, strong winds, or blocked roads. Since the drone receives only partial updates, it may choose a risky route or need to search again when new information arrives.

ii. How A* Balances Path Cost and Heuristic

A* evaluates each possible route using $f(n) = g(n) + h(n)$. Here, $g(n)$ is the actual cost from the starting point to the current location, including travel time, terrain difficulty, and weather-related cost. The heuristic $h(n)$ estimates the remaining distance or cost to the destination. By considering both values, A* avoids relying only on the apparently shortest route. If the heuristic is admissible and the path costs are represented correctly, A* can find an optimal route in a static search problem. In this changing environment, the drone must update costs and replan when it receives new information.

iii. Impact of Heuristic Accuracy

A good heuristic gives a useful estimate of how far the destination is and helps the search explore fewer unnecessary locations. If the straight-line distance is close to the real travel cost, both algorithms can be guided toward useful routes. However, straight-line distance does not show flooding, wind, or terrain conditions. Greedy Best-First Search is especially affected because it relies mainly on the heuristic. A* is generally more balanced because it also considers the cost already incurred, although a poor heuristic can still make it slower or lead to inefficient exploration.

iv. Effect of Dynamic Blocked Paths

When a route becomes blocked, the drone's previous plan may no longer be usable. The system must incorporate the update, remove or penalize the affected path, and search for another route. Repeated changes can increase computation and delay delivery. A practical system should use current sensor and weather information and replan only when needed. Algorithms designed for changing maps, such as incremental or real-time replanning methods, can reduce repeated work.

v. Recommended Algorithm

A* is the better basic choice when the drone needs to balance travel cost, delay, and risk because it considers both the route already travelled and the estimated remaining cost. The route cost should include safety factors such as wind, flooding, battery use, and restricted areas, rather than distance alone. Since the environment changes, A* should be combined with frequent map updates and rapid replanning. If immediate action is essential, the drone can use a safe provisional route while

continuing to improve its plan. This gives a reasonable balance between efficiency, optimality, and safety.

2. Smart-City Traffic Signal Optimization

Formulation as an Optimization Problem

The objective is to choose signal timings across intersections that minimize total vehicle waiting time, fuel consumption, and congestion. The decision variables can include green-light duration, cycle length, and coordination between nearby signals. The objective function can be written as a weighted combination of these three measures, subject to constraints such as minimum and maximum green times, pedestrian crossing requirements, and safe phase ordering. Traditional exhaustive search is inefficient because the number of possible timing combinations grows rapidly with the number of intersections. Traffic conditions also change continuously, so a solution may become outdated quickly.

i. How Hill Climbing Works and Its Limitations

Hill Climbing starts with an initial signal-timing plan and evaluates nearby alternatives. It moves to a neighbouring plan whenever that plan improves the objective, repeating the process until no better neighbour is found. It is simple and can improve a solution quickly, which is useful for a large traffic network. Its limitation is that it normally accepts only immediate improvements. It may stop at a locally good timing plan even when a much better plan exists elsewhere in the search space.

ii. Local Maxima, Plateaus and Ridges

In this minimization problem, the equivalent of a local maximum is a local optimum: a timing plan that is better than its nearby alternatives but not the best overall. A plateau occurs when many nearby plans have almost the same performance, so the algorithm has no clear direction. A ridge occurs when improvement requires a sequence of coordinated changes across multiple signals, while changing only one signal at a time may not help. In real traffic, one intersection's timing can affect queues at neighbouring intersections, so optimizing each signal independently may increase congestion elsewhere.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse neighbouring solution early in the search, with a probability controlled by a temperature value. As the temperature decreases, the algorithm becomes less likely to accept worse solutions. This helps it escape local optima and explore a wider range of timing plans. A Genetic Algorithm maintains a population of candidate plans and creates new candidates through selection, crossover, and mutation. It can explore different combinations of signal timings at once. Both methods can search beyond immediate improvements, though their performance depends on suitable parameter settings and a well-designed objective function.

iv. Recommended Approach

A Genetic Algorithm is a suitable choice for optimizing timings across many connected intersections because a candidate solution can represent the full network, allowing coordinated changes. The objective should measure waiting time, fuel use, and congestion, while enforcing safety and pedestrian constraints. To handle changing traffic, the system can update its input data and periodically optimize the population, using the current best plan as a starting point. Simulated Annealing may be simpler for smaller optimization problems or as a local improvement step. In practice, a hybrid approach with fast local adjustments and periodic wider optimization can offer a useful balance of scalability and adaptability.

3. Autonomous Mars Rover Exploring Unknown Terrain

i. Why an Online Search Agent Is Needed

Offline search assumes that the agent has enough information about the environment to plan before acting. A Mars rover does not have a complete map of every rock, crater, slope, or obstacle. It must move, collect sensor readings, update its knowledge, and decide what to do next. Online search is therefore more appropriate because planning and action take place together. The rover can choose a safe next move based on what it currently knows and adjust its plan when it discovers new terrain.

ii. Partial Observability and Dynamic Conditions

The rover's sensors provide only a limited view of the environment, and some hazards may be hidden or difficult to measure. Sensor noise, communication delays, limited energy, and uncertain terrain can make decisions difficult. A route that looks safe from a distance may contain a steep slope or loose soil. Although the terrain itself may not change quickly, the rover's understanding of it changes as new observations become available. The rover must therefore account for uncertainty, avoid unnecessary risks, and keep enough resources for recovery or communication.

iii. Building and Updating an Internal Model

The rover builds an internal model by combining its starting map, camera images, range measurements, wheel and movement sensors, and other available observations. It records known safe areas, obstacles, possible paths, and locations worth investigating. After each movement, it compares the expected result with the actual sensor readings and updates its map and estimates. Unknown areas can be marked as uncertain rather than automatically treated as safe. The rover then uses the revised model to select its next action.

iv. Online Search Compared with Uninformed and Informed Search

Uninformed search methods, such as breadth-first search, explore without using a goal-distance estimate. They can be useful in simple settings but may waste time exploring many irrelevant locations and often assume the state space is already available. Informed search uses a heuristic to guide exploration toward a goal, which can reduce unnecessary work. Online search differs because

the agent learns about the environment while acting and cannot rely on a complete map in advance. An informed online method can combine goal guidance with continual observation and replanning.

v. Most Suitable Approach

A safe online search agent using heuristic guidance is most suitable for the rover. It should choose actions using the known map and estimated distance to the target, then update the map after every important observation. Safety rules should reject paths with unacceptable slope, obstacle, or energy risks, even if those paths appear short. The rover should also have a fallback plan when sensing is uncertain or communication is lost. This approach is adaptable because it can respond to discoveries without requiring a complete map before the mission begins.

4. University Exam Timetabling as a Constraint Satisfaction Problem

CSP Formulation: Variables, Domains and Constraints

The exam timetable can be represented as a Constraint Satisfaction Problem (CSP), which consists of variables, domains, and constraints. Each course exam is a variable. Its domain is the set of available time slots and exam halls in which it could be scheduled. Constraints define which combinations are allowed. For example, two exams taken by the same student cannot occur at the same time, the number of exams assigned to a hall cannot exceed its capacity, and a subject that must come before another must be scheduled earlier. Faculty availability and special accommodations must also be included.

ii. How Backtracking Search Works

Backtracking search assigns a time slot and hall to one exam at a time. After each assignment, it checks whether the choice conflicts with any constraints. If the assignment is valid, the algorithm moves to the next exam. If it creates a conflict or a later exam has no valid option, the algorithm reverses the most recent assignment and tries another choice. This process continues until a complete valid timetable is found or all possible assignments have been considered. Choosing the most constrained exam first can reduce unnecessary backtracking.

iii. How Constraint Propagation Improves Efficiency

Constraint propagation reduces the available choices before the algorithm explores them deeply. With forward checking, after an exam is assigned, the algorithm removes incompatible time slots or halls from the domains of related exams. If any unassigned exam has no remaining option, the algorithm detects the problem immediately and backtracks. This prevents the system from continuing along a branch that cannot lead to a valid timetable. Propagation can be combined with variable-ordering methods, such as choosing the exam with the fewest legal options first.

iv. Real-World Challenges and Scalable Strategy

A real university may have thousands of students, many courses, limited halls, different room capacities, faculty schedules, and accessibility needs. Some constraints are absolute, while preferences such as avoiding consecutive exams may be soft constraints that should be minimized

rather than treated as impossible to violate. A practical strategy is to use CSP backtracking with forward checking and a minimum-remaining-values heuristic, while ordering choices to satisfy the most restrictive constraints early. For large instances, the system can use optimization methods or divide the timetable into manageable groups, while checking that the final timetable remains globally consistent.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax Models the Game

Minimax represents the game as a tree of possible states and moves. The AI is treated as the maximizing player, trying to achieve the highest evaluation score, while the human opponent is treated as the minimizing player, trying to reduce that score. At terminal states, the algorithm assigns a utility value for outcomes such as winning, losing, or drawing. These values are passed upward through the tree: maximizing levels select the highest value, and minimizing levels select the lowest. The AI chooses the move with the best result assuming the opponent also makes strong decisions.

ii. Computational Challenges of Large Game Trees

If each state has an average of b possible moves and the search looks ahead d levels, the number of nodes can grow roughly as O(b^d). Even a moderate branching factor becomes expensive at greater depths. In a real-time strategy game, many units, resources, and possible actions create a large number of branches, and the AI may not have enough time to explore them all. A deeper search can improve decisions but may miss the response deadline. The system therefore needs to limit the search and estimate the value of states that are not fully explored.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning improves Minimax by skipping branches that cannot affect the final choice. Alpha is the best value found so far for the maximizing player, and beta is the best value found so far for the minimizing player. When alpha becomes greater than or equal to beta, the remaining branches at that point can be pruned because they cannot improve the decision for the player whose choice matters. The result is the same as standard Minimax when the same terminal evaluations and search depth are used, but fewer nodes may need to be examined. Good move ordering makes pruning more effective and can allow the AI to search deeper within the same time limit.

iv. Designing an Evaluation Function

A practical evaluation function can combine several weighted factors: the difference in remaining units, control of important locations, available resources, progress toward objectives, defensive strength, and immediate threats. For example, a higher score can be given for having more surviving units, controlling strategic areas, and maintaining sufficient resources, while penalties can be applied for exposed units or threatened bases. The weights should reflect the game's rules and be tested against different playing styles. The function should also recognize urgent dangers so that the AI does not choose a high long-term score while ignoring an immediate loss.